# hypercluster

## End-to-End Kubernetes Lifecycle Management

Kubespray-powered · SSH-first · Podman-native

# The Problem

Deploying a production-grade Kubernetes cluster is still hard:

| Pain Point | Reality |
| --- | --- |
| Tool sprawl | kubeadm + Ansible + Helm + scripts |
| Inventory hell | YAML by hand, per-cluster, error-prone |
| Day-2 forgotten | Scaling and upgrades need separate runbooks |
| kubeconfig chaos | Scattered, unpatched files |
| macOS gap | Cloud providers; no local testing story |

> There is no single command that takes you from bare nodes to a running cluster and handles the full lifecycle afterward.

# The Solution: hypercluster

**One config file. One binary. Full lifecycle.**

```
cluster.conf  ──►  hypercluster  ──►  Kubespray (Ansible)  ──►  nodes via SSH
                        │
                        ├─ clones Kubespray automatically
                        ├─ generates hosts.yaml + group_vars
                        ├─ runs cluster.yml / scale.yml / upgrade_cluster.yml
                        └─ fetches and patches kubeconfig locally
```

hypercluster is a **thin shell wrapper** — it does no Kubernetes work itself.
Every operation delegates to a Kubespray Ansible playbook.

# Architecture at a Glance

```
hypercluster/
├── hypercluster          # CLI entry point (bash)
├── cluster.conf          # Single config file you edit
├── lib/
│   ├── log.sh            # Emoji-based logging
│   ├── preflight.sh      # Dependency + SSH checks
│   ├── inventory.sh      # hosts.yaml + group_vars generation
│   ├── kubespray.sh      # Clone, version-lock, run playbooks
│   └── kubeconfig.sh     # Fetch, patch, merge kubeconfig
├── scripts/
│   ├── k8s-deploy.sh     # Standalone k3s/kubeadm deployer
│   ├── test-podman.sh    # Local testing via Podman containers
│   └── setup-macos.sh    # One-shot macOS setup
└── config/
    └── cluster.conf.example
```

# Supported Operations

| Command | What it does |
| --- | --- |
| `hypercluster init` | Clone Kubespray, generate inventory |
| `hypercluster preflight` | Check SSH, dependencies, key permissions |
| `hypercluster deploy` | Full cluster bootstrap |
| `hypercluster scale` | Add worker nodes |
| `hypercluster upgrade` | Rolling Kubernetes version upgrade |
| `hypercluster remove-node` | Gracefully drain and remove a node |
| `hypercluster fetch-kubeconfig` | Pull kubeconfig, patch IP, merge |
| `hypercluster status` | `kubectl get nodes` via fetched kubeconfig |
| `hypercluster destroy` | Tear down the cluster |

# One Config File

```
# cluster.conf
CLUSTER_NAME="prod-k8s"
KUBESPRAY_VERSION="v2.30.0"

KUBE_VERSION="v1.32.0"
CONTAINER_MANAGER="podman"      # podman | containerd | docker
CNI_PLUGIN="calico"             # calico | cilium | flannel

CONTROL_PLANE_IPS="10.0.1.10"
WORKER_IPS="10.0.1.11 10.0.1.12 10.0.1.13"

SSH_USER="root"
SSH_KEY="$HOME/.ssh/id_cluster"
SSH_PORT=22

METRICS_SERVER_ENABLED="true"
KUBECONFIG_OUTPUT="$HOME/.kube/prod-k8s.yaml"
```

# Deploy in 3 Steps

## Step 1 — Copy and edit config

```
cp config/cluster.conf.example cluster.conf
# edit cluster.conf with your node IPs and SSH key
```

## Step 2 — Preflight check

```
./hypercluster preflight
# ✅  ansible: ansible [core 2.18.1]
# ✅  SSH 10.0.1.10 root reachable
# ✅  SSH key permissions OK
```

## Step 3 — Deploy

```
./hypercluster deploy
# Clones Kubespray, builds inventory, runs cluster.yml
# 🎉  Cluster ready!  export KUBECONFIG=$HOME/.kube/prod-k8s.yaml
```

# Inventory Generation — Automatic

hypercluster generates `hosts.yaml` from your IP list:

```yaml
# auto-generated — do not edit by hand
all:
  hosts:
    master1:
      ansible_host: 10.0.1.10
      ip: 10.0.1.10
    worker1:
      ansible_host: 10.0.1.11
      ip: 10.0.1.11
  children:
    kube_control_plane:
      hosts: { master1: }
    kube_node:
      hosts: { worker1: }
    etcd:
      hosts: { master1: }
```

# Podman as Container Runtime

hypercluster defaults to **Podman** — the production-ready, daemonless OCI runtime:

| Feature | Podman | Docker |
|---|---|---|
| Daemon required | No | Yes |
| Rootless support | Yes | Partial |
| systemd integration | Native | Plugin |
| OCI compliance | Full | Full |
| RHEL / Rocky default | Yes | No |

Kubespray config generated automatically:

```
container_manager: podman
kubelet_cgroup_driver: systemd
```

# Day-2: Scale Without Downtime

Add nodes by editing `cluster.conf` and running scale:

```
# Add to cluster.conf:
WORKER_IPS="10.0.1.11 10.0.1.12 10.0.1.13 10.0.1.14"

./hypercluster scale
# ✅ Regenerating inventory with new node
# ✅ Running scale.yml playbook
# ✅ Node 10.0.1.14 joined the cluster
```

Kubespray's `scale.yml` only touches the new node —
**existing workloads are not disrupted.**

# Day-2: Zero-Downtime Upgrade

```
# In cluster.conf, bump:
KUBE_VERSION="v1.33.0"

./hypercluster upgrade
# ✅  Running upgrade_cluster.yml
# Rolling upgrade: masters first, then workers
# Existing pods are rescheduled, not deleted
```

Kubespray performs a **rolling upgrade** — one node at a time, draining workloads before upgrading each node.

# kubeconfig — Fetch and Merge

```
./hypercluster fetch-kubeconfig
# ✅  Artifact found: .kubespray/inventory/prod-k8s/artifacts/admin.conf
# ✅  Patched 127.0.0.1 → 10.0.1.10
# ✅  Merged into ~/.kube/config
# ✅  kubeconfig saved → ~/.kube/prod-k8s.yaml

export KUBECONFIG=~/.kube/prod-k8s.yaml
kubectl get nodes
# NAME      STATUS   ROLES           AGE   VERSION
# master1   Ready    control-plane   5m    v1.32.0
# worker1   Ready    <none>          4m    v1.32.0
```

# Standalone Script: k8s-deploy.sh

**No Kubespray, no Ansible** — pure bash k3s/kubeadm deployer:

```
# Edit the CONFIGURATION section, then:
./scripts/k8s-deploy.sh

# Supports:
# - k3s (lightweight, single binary)
# - kubeadm (upstream, production-grade)
# - Podman / Docker / none runtime
# - Parallel worker installation
# - Auto kubeconfig fetch + merge
```

Override config without editing the script:

```
HC_CONFIG=/path/to/my.conf ./scripts/k8s-deploy.sh
```

# Local Testing on macOS — Podman Containers

No cloud, no VMs, no sudo required:

```
# One-time setup
./scripts/setup-macos.sh

# Start 3 systemd-enabled containers (1 master + 2 workers)
# Deploy k3s, verify cluster — all locally:
./scripts/test-podman.sh

# Preflight only (SSH connectivity check):
./scripts/test-podman.sh --preflight

# Tear down:
./scripts/test-podman.sh --clean
```

Each container runs **Ubuntu 22.04 with systemd as PID 1** — identical to a real node's service management.

# macOS Setup — One Command

```
./scripts/setup-macos.sh
```

Installs via Homebrew:

- **bash 5** (macOS ships 3.2 — too old for associative arrays)
- **ansible** (not pip — avoids PEP 668 "externally managed" error)
- **kubectl**, **helm**
- **podman** + machine init

Python modules (jinja2, netaddr) installed with:

```
pip3 install --user --break-system-packages jinja2 netaddr
```

Updates `~/.zshrc` / `~/.bash_profile` with Homebrew PATH.

# Security Model

- **SSH key auth only** — no passwords, no agents on nodes
- **No cloud provider dependency** — works air-gapped
- **Minimal permissions** — runs as your user, SSH to root on nodes
- **Lock file** prevents concurrent runs on the same cluster
- **kubeconfig chmod 600** enforced on fetch

Preflight checks:

```
✅ SSH key permissions: 600 (OK)
✅ ansible-playbook reachable
✅ python3 available on all nodes
✅ kubectl installed locally
```

# Add-ons via cluster.conf

```
METRICS_SERVER_ENABLED="true"     # kubectl top nodes/pods
INGRESS_NGINX_ENABLED="false"     # Ingress controller
DASHBOARD_ENABLED="false"         # Kubernetes Dashboard
```

Kubespray handles add-on installation as part of `cluster.yml`.

Toggle and re-run `deploy` to apply changes.

# Comparison

| | hypercluster | Raw Kubespray | kubeadm | k3s |
|---|---|---|---|---|
| Config complexity | Single file | Multi-YAML | Multi-step | Env vars |
| Day-2 ops | Built-in | Manual playbooks | Manual | Manual |
| kubeconfig mgmt | Auto | Manual | Manual | Manual |
| macOS testing | Podman | Multipass/VM | VM | Possible |
| Production grade | Yes | Yes | Yes | Lightweight |
| Learning curve | Low | Medium | High | Low |

# Roadmap

- **Multi-master HA** — multiple control-plane IPs in config
- **Vault integration** — encrypted secrets in cluster.conf
- **GitOps bootstrap** — ArgoCD / Flux install post-deploy
- **Node health checks** — pre-flight disk/memory/CPU thresholds
- **Windows nodes** — kubeadm hybrid clusters
- **Web UI** — optional dashboard for cluster status

# Get Started

```
git clone <your-repo>/hypercluster
cd hypercluster

# macOS?
./scripts/setup-macos.sh

# Test locally (no cloud needed):
./scripts/test-podman.sh

# Real nodes:
cp config/cluster.conf.example cluster.conf
# edit cluster.conf
./hypercluster deploy
```

**hypercluster** — from bare nodes to running cluster,
and every operation after that.